08

Scaffold: o terreno antes do código

O agent lê o workspace inteiro como parte do prompt — não só as mensagens do chat. Um diretório vazio é, portanto, um prompt vago. O scaffold é a estrutura inicial criada antes de qualquer código de negócio: pastas, configurações, dependências instaladas e um servidor mínimo de pé. É o terreno onde as tasks do PLAN.md vão acontecer.

A IDEIA CENTRAL DESTE MATERIAL

Context File + scaffold + um teste passando levam 30 a 60 minutos — e mudam o patamar de consistência de tudo que vem depois. Sem scaffold, cada agent em paralelo inventa a própria estrutura; com scaffold, todos trabalham no mesmo terreno.

1 Por que antes da implementação

Quando o scaffold existe antes de qualquer código, o agent já conhece a stack, enxerga a estrutura de componentes e entende o modelo de deployment — tudo sem você explicar nada.

Sem ele, o problema clássico do paralelismo: um agent cria `/src/services/`, outro cria `/app/services/`, o terceiro cria `/lib/services/` — e nenhum consegue importar código do outro. O scaffold elimina isso definindo a estrutura **uma vez, antes de qualquer execução**.

Há um bônus menos óbvio: escrever o Context File antes do código força decisões **deliberadas** sobre convenções — em vez de deixar que os primeiros arquivos gerados as definam por acidente.

2 Scaffold não é a Fase 1 do PLAN.md

A distinção mais importante — e mais ignorada:

	SCAFFOLD	FASE 1 DO PLAN.MD
Output	Pastas + config + servidor vazio	Modelos, schemas, configurações de negócio
Agents	Um único, sequencial	Múltiplos em paralelo
Critério	Servidor sobe sem erro	Testes da fase passam
Quando termina	Antes de qualquer implementação	Antes da Fase 2 começar

O scaffold é o **pré-requisito** da Fase 1: é ele que garante que as tasks paralelas da Fase 1 encontram a mesma estrutura.

3 O que o scaffold cria — e o que não cria

✓ CRIA

Estrutura de pastas — com um README.md de uma linha em cada diretório descrevendo sua responsabilidade.

Configurações na raiz — `tsconfig.json`, linter, formatter: os primeiros arquivos que o agent lê para inferir o setup.

Dependências instaladas — não apenas declaradas.

Ponto de entrada mínimo — servidor com uma única rota `/health` retornando 200.

Setup de testes — framework configurado, pasta espelhando a estrutura, um smoke test passando.

✗ NÃO CRIA

Nenhuma lógica de negócio.

Nenhum modelo de dados ou schema.

Nenhum endpoint além de `/health`.

Nenhum componente de UI além do layout base vazio.

Nenhum teste de feature — só o smoke test.

E não toca em SPEC.md, CLAUDE.md, PLAN.md, .claudeignore — eles já existem e permanecem intactos.

4 Critério de conclusão

O scaffold está completo quando dois comandos retornam sucesso — e **nenhuma task de implementação começa antes disso**:

```
# Backend
curl localhost:8000/health
→ {"status": "ok"}

# Frontend
curl localhost:3000
→ HTML da página base, sem erro
```

5 Padrões de estrutura por stack

Cada stack tem um padrão estabelecido. O essencial: o padrão é definido no **CLAUDE.md/AGENTS.md antes do scaffold rodar** — não decidido durante.

```
# Python + FastAPI
backend/
├── app/
│   ├── main.py
│   ├── core/
│   ├── models/
│   ├── schemas/
│   ├── services/
│   └── api/
└── tests/

# Node + Express/Fastify
backend/
├── src/
│   ├── index.ts
│   ├── config/
│   ├── models/
│   ├── services/
│   ├── routes/
│   └── middleware/
└── tests/

# Next.js
frontend/
├── src/
│   ├── app/ (layout.tsx, page.tsx)
│   ├── components/
│   ├── lib/
│   └── types/
└── __tests__/
```

6 Como cada ferramenta scaffolda

FERRAMENTA	ABORDAGEM
Claude Code	`/init` gera o CLAUDE.md analisando o repositório; para projeto novo, o agent cria a estrutura e instala dependências via terminal.
OpenCode	Mesmo padrão via `/init` (gera AGENTS.md), seguido de instruções explícitas de estrutura.
Codex CLI	Scaffold (estrutura, CI, formatação, package manager) gerado antes de qualquer feature, guiado por templates.
Cursor / Windsurf	O agent cria a estrutura via terminal integrado, com aprovação por diff antes de cada arquivo.
Gemini CLI	Executa os comandos lendo o GEMINI.md para entender as convenções antes de scaffoldar.

7 Erros mais comuns

ERRO	CONSEQUÊNCIA

Scaffold com código de negócio ("aproveitei o embalo")	A Fase 1 encontra código incompleto fora das convenções — e herda o problema.
Dependências declaradas mas não instaladas	A Fase 1 quebra no primeiro import.
Estrutura divergente do PLAN.md	O agent cria `/src/auth/`, mas o plano referencia `/app/api/v1/auth.py` — as tasks não sabem onde colocar os arquivos.
Scaffold sem smoke test	O servidor sobe, a primeira task quebra o import e ninguém percebe até a Fase 2.
Pular o scaffold e ir direto para a Fase 1	Agents gastam tokens criando estrutura em vez de implementar — e cada um cria uma diferente. O ambiente subespecificado é o que torna o progresso "misteriosamente" lento.

8 Prompt pronto: gerando o scaffold

▸ PROMPT PARA GERAR O SCAFFOLD

```
Leia SPEC.md, CLAUDE.md (ou AGENTS.md) e PLAN.md na raiz do projeto.
Gere o scaffold completo seguindo estas regras:

1. ESTRUTURA DE PASTAS
   Crie todos os diretórios definidos no CLAUDE.md/AGENTS.md,
   com um README.md de uma linha descrevendo cada pasta.

2. ARQUIVOS DE CONFIGURAÇÃO
   Crie todos os arquivos da stack (requirements.txt, package.json,
   tsconfig.json, .env.example, .gitignore etc.)
   e INSTALE as dependências após criar.

3. PONTO DE ENTRADA MÍNIMO
   main.py / index.ts (conforme a stack) apenas com o servidor
   inicializado e a rota /health retornando 200.
   Nenhuma lógica de negócio.

4. SETUP DE TESTES
   Framework definido no CLAUDE.md, pasta de testes espelhando
   a estrutura, um smoke test verificando que o servidor sobe.

5. O QUE NÃO FAZER
   - Nenhuma lógica de negócio, modelo de dados ou endpoint
     funcional além de /health
   - Nenhum componente de UI além do layout base

6. CRITÉRIO DE CONCLUSÃO
   Execute os comandos de verificação do CLAUDE.md e confirme
   que o servidor sobe sem erros antes de encerrar.
```

▸ COLA PARA FIXAR

- O agent lê o **workspace como prompt**: diretório vazio = prompt vago.
- Scaffold = pastas + configs + dependências instaladas + `/health` + smoke test. **Zero negócio.**
- Scaffold é **pré-requisito da Fase 1**, feito por um único agent — o paralelismo começa depois.
- Critério objetivo: dois `curl` retornando sucesso. Antes disso, nenhuma task começa.
- O padrão de estrutura é decidido no **CLAUDE.md, antes** — não durante o scaffold.
- 30–60 minutos de terreno preparado economizam horas de estrutura divergente.